# Práctica de repaso · Mi agenda de tareas

**Módulo:** MF0491_3 · Programación web en el entorno cliente
**Tecnologías:** HTML, CSS y JavaScript
**Modalidad:** Práctica guiada

## Tu misión: organiza tu día

Vamos a crear una aplicación para organizar nuestras tareas diarias.

La aplicación permitirá registrar actividades, consultar las tareas pendientes y calcular cuánto tiempo necesitamos para completarlas.

Ya tienes preparados `index.html` y `styles.css`. Tu trabajo consiste en programar `app.js` y realizar algunas modificaciones en HTML y CSS.

**Trabaja los retos en orden y comprueba cada uno antes de avanzar.**

---

## RETO 1 · Crear y mostrar las tareas

**Objetivo:** mostrar las tareas de nuestra agenda utilizando JavaScript.

### Paso 1. Crear el array

En `app.js`, crea un array llamado `tareas` que contenga dos objetos con estas propiedades:

| Nombre | Categoría | Minutos |
|---|---|---|
| Estudiar JavaScript | Estudio | 45 |
| Hacer ejercicio | Personal | 30 |

**Pista:** cada elemento del array es un objeto con tres propiedades: `nombre`, `categoria` y `minutos`.

### Paso 2. Mostrar las tareas

1. Selecciona el contenedor `#listaTareas` del HTML.
2. Crea una función llamada `mostrarTareas()`.
3. Limpia el contenedor antes de mostrar las tareas.

4. Recorre el array con `for...of`.
5. Crea una tarjeta por cada tarea mostrando su nombre, categoría y duración.
6. Ejecuta `mostrarTareas()` al abrir la página.

Puedes utilizar `innerHTML` o `createElement()`.

**Pistas:** `querySelector()`, `for...of`, `innerHTML`, `textContent`.

En el CSS ya existen las clases `.tarea` y `.duracion`.

**Comprueba:** al abrir la página aparecen las dos tareas con sus datos.

---

# RETO 2 · Añadir nuevas tareas

**Objetivo:** registrar actividades desde el formulario.

### Paso 1. Validar desde HTML

Abre `index.html` y añade los atributos necesarios para que:

- El nombre de la tarea sea obligatorio.
- Sea obligatorio seleccionar una categoría.
- La duración sea obligatoria y no pueda ser negativa.

**Pistas:** `required`, `min`.

### Paso 2. Programar el formulario

En `app.js`:

1. Selecciona el formulario y sus tres campos.
2. Crea una función `agregarTarea()`.
3. Asocia la función al evento `submit` y evita que la página se recargue.
4. Lee los valores introducidos y convierte los minutos a número.
5. Crea un objeto `nuevaTarea` con `nombre`, `categoria` y `minutos`.
6. Añade el objeto al array mediante `push()`.
7. Actualiza el listado, limpia el formulario y muestra un mensaje de confirmación.

**Pistas:** `.value`, `Number()`, `push()`, `addEventListener("submit", ...)`, `preventDefault()`, `reset()`.

### Paso 3. Contar las tareas

Selecciona `#totalTareas` y modifica `mostrarTareas()` para actualizar el contador utilizando la propiedad `length`.

**Comprueba:**

Añade esta actividad:

| Nombre | Categoría | Minutos |
|---|---|---|
| Preparar la mochila | Personal | 15 |

Deben aparecer **3 tareas**.

Prueba también a enviar el formulario con un campo vacío. El navegador debe impedirlo.

---

# RETO 3 · Calcular el tiempo total

**Objetivo:** calcular cuántos minutos necesitaremos para realizar todas las tareas.

El HTML ya contiene el elemento `#tiempoTotal`.

## Instrucciones

1. Selecciona `#tiempoTotal`.
2. Crea una función llamada `calcularTiempo()`.
3. Declara una variable `total` que comience en 0.
4. Recorre el array `tareas` y suma los minutos de cada actividad.
5. Muestra el resultado acompañado de la palabra «minutos».
6. Actualiza el resultado cada vez que se añada una tarea.

**Pistas:** `for...of`, `+=`, `textContent`.

**Comprueba:**

- Estudiar JavaScript (45) + Hacer ejercicio (30) = **75 minutos**.
- Al añadir Preparar la mochila (15), el total debe ser **90 minutos**.

---

# RETO 4 · El rincón de la productividad

**Objetivo:** añadir contenido nuevo a la página y hacerlo interactivo con JavaScript.

## Paso 1. HTML

Después del listado de tareas, crea una sección titulada:

**El rincón de la productividad**

Incluye dos botones:

- «Consejo para organizarte».
- «Mensaje motivador».

Añade un párrafo vacío donde aparecerán los mensajes. Asigna identificadores a los elementos que necesites.

### Paso 2. CSS

Añade estilos a la nueva sección y sus botones.

Debe tener un fondo diferente, separación entre los elementos y un efecto `:hover` en los botones.

### Paso 3. JavaScript

1. Selecciona los botones y el párrafo.
2. Crea una función para mostrar los mensajes.
3. Utiliza eventos `click` para ejecutar las acciones.
4. Muestra el mensaje correspondiente mediante `textContent`.

**Botón «Consejo para organizarte»:**

«Empieza por una tarea sencilla y ve avanzando».

**Botón «Mensaje motivador»:**

«¡Una tarea cada vez y cada vez más cerca de tu objetivo!».

**Pistas:** `querySelector()`, `onclick` o `addEventListener()`, `if`, `textContent`.

**Comprueba:** ambos botones muestran correctamente su mensaje sin recargar la página.

---

## RETO EXTRA · Guardar las tareas

**Objetivo:** conseguir que las tareas permanezcan guardadas al recargar la página.

1. Crea una función `guardarTareas()` que convierta el array a JSON y lo guarde en `localStorage`.
2. Crea una función `cargarTareas()` para recuperar los datos guardados.
3. Guarda las tareas después de añadir una nueva.

4. Recupera las tareas al iniciar la aplicación, antes de mostrarlas.

Utiliza la clave `"agenda_tareas"`.

**Pistas:** `JSON.stringify()`, `JSON.parse()`, `localStorage.setItem()`, `localStorage.getItem()`.

**Comprueba:** añade una tarea, recarga la página y verifica que sigue apareciendo y que el tiempo total se mantiene actualizado.

---

# Preguntas para comprobar lo aprendido

Antes de terminar, intenta responder:

1. ¿Por qué utilizamos un array de objetos para guardar las tareas?
2. ¿Qué hace `push()` y por qué necesitamos ejecutar después `mostrarTareas()`?
3. ¿Qué conseguimos utilizando `preventDefault()`?
4. ¿Por qué convertimos los minutos a número antes de sumarlos?
5. ¿Qué diferencia existe entre los datos del array y los elementos que mostramos en el HTML?
6. ¿Para qué sirven `JSON.stringify()` y `JSON.parse()`?

# Comprobación final

- Se muestran las tareas iniciales.
- Puedo añadir nuevas tareas.
- El formulario comprueba los campos obligatorios.
- El contador se actualiza.
- El tiempo total se calcula correctamente.
- Los botones muestran sus mensajes.
- No aparecen errores en la consola.

**Recuerda:** puedes consultar los apuntes y utilizar el autocompletado. Antes de escribir código, piensa qué información necesitas, dónde se encuentra y qué resultado quieres conseguir.

**¡Ya estás preparado para aplicar estos conocimientos a tu Gestor de citas!**